

SKRoot(Pro) 模块开发指南

一、SKRoot 模块介绍

SKRoot(Pro) 模块开发风格接近 Linux 内核模块，简单易上手。主要具备以下三大核心优势：

- 1. 双态切换执行：**支持用户态（EL0）与内核态（EL1）之间自由切换执行。通过 SDK 接口可以直接读写内核内存、调用内核函数。
- 2. 一次编译，全线通用：**内核偏移由 SDK 统一封装提供，一次编译即可跨所有内核通用（Linux 3.10 ~ 6.18 环境）。
- 3. 自研内核 Hook 框架：**0 性能损耗、无侧信道痕迹。提供安全、隐蔽的内核 Hook 能力。

二、SKRoot 模块运行原理

2.1 模块整体架构

- 2.1.1 SKRoot 模块本质上是一个运行于 Android 用户态（EL0）的动态链接库。通过静态链接 **SKRoot SDK**，能获得访问内核的接口，从而建立了一条用户态程序与内核态之间的交互通道

2.2 模块加载流程

- 2.2.1 SKRoot 模块以 Android JNI **.so** 动态库形式存在，由 **Root 权限进程**通过 **dlopen()** 动态加载。

- **开机自启动方式：**模块会在系统启动早期阶段，在 zygote64 启动前完成加载。
- **现场漏洞提权方式：**系统启动完成后，通过 CVE 漏洞等方式在运行环境中获取 Root 权限，并在获得权限后开始加载模块。

2.2.2 加载完成后，SKRoot 模块仍然运行于 Android 用户态 (EL0)，其执行环境与普通用户态动态库类似。

三、 基础能力

- 3.1 用户层 Root 权限代码执行
- 3.2 读写内核内存
- 3.3 执行内核态代码
- 3.4 调用内核函数
- 3.5 安装内核 Hook

四、 内核偏移与跨内核适配

- 4.1 跨内核偏移获取（包含结构体内偏移）
- 4.2 跨内核物理地址计算
- 4.3 跨内核函数调用

五、 SDK 头文件接口说明

5.1 基础接口说明

序号	头文件	说明
1	kernel_module_kit_umbrella.h	一键包含全部头文件。
2	module_base_kernel_mem.h	读写内核内存
3	module_base_kernel_func_executor.h	执行内核态代码
4	module_base_kernel_func_hook.h	安装内核 Hook
5	module_base_kernel_symbol_kaddr.h	查找内核符号地址
6	module_base_kernel_export_symbol.h	内核导出符号调用封装
7	module_base_kernel_string_ops.h	内核字符串与内存操作封装
8	module_base_kernel_btf_resolver.h	内核 BTF 解析器（动态偏移补全）
9	module_base_disk_storage.h	磁盘持久化存储
10	module_descriptor.h	模块名片信息
11	module_web_ui_http_handler.h	模块 Web 管理页面基类
12	module_base_install_callback.h	模块安装与卸载回调
13	module_base_change_status.h	模块动态修改描述

5.2 跨内核偏移接口说明

子文件夹名称	kernel_offsets/
结构体偏移头文件	address_space.h block_device.h cred.h dentry.h fdtable.h file.h file_operations.h file_struct.h gendisk.h inode.h inode_operations.h miscdevice.h mm_struct.h page.h proc_ops.h pte_fn_t.h renamedata.h seq_file.h seq_operations.h super_block.h task_struct.h vm_area_struct.h kstat.h

**跨内核偏移获取手段：通过“用户层旁敲侧击佐证 + 内核结构模型相似度打分”实现，并非使用 `ebpf` 等内核接口。*

**支持跨机型、跨内核 (Linux3.10~6.18)。*

六、演示工程。

6.1 开发基础：首个 SKRoot 模块

6.1.1 引入核心头文件 `kernel_module_kit_umbrella.h`

6.1.2 实现模块入口 `skroot_module_main` 与模块名片

6.2 Hello World 演示

源码路径：testModule/`module_hello_world`

6.3 内置 WebUI 演示

源码路径：testModule/`module_web_ui_example` 目录

6.4 安装/卸载回调演示

源码路径：testModule/ `module_install_uninstall_cb_demo` 目录

6.5 URL 配置更新演示

源码路径：testModule/ `module_update_demo` 目录

6.6 系统文件伪造演示

源码路径：testModule/ `module_fake_system_file` 目录

七、编译环境。

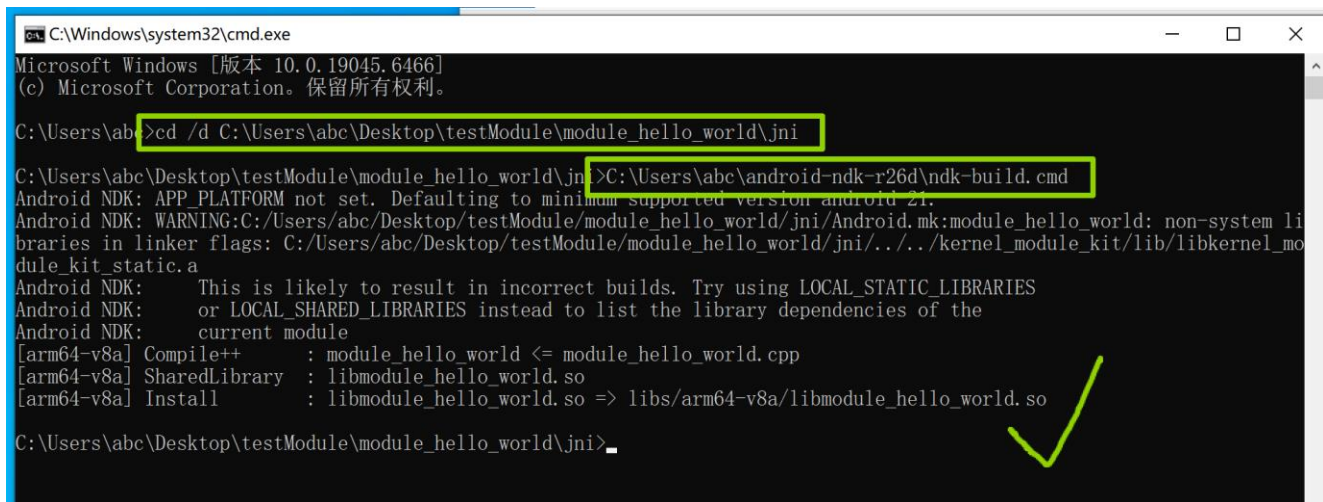
7.1 需要 Android NDK 26d 及以上。

7.2 演示 Windows 编译 Hello World 工程。

7.2.1 打开 cmd，输入 `cd /d C:\xxx\testModule\module_hello_world\jni` 回车

7.2.2 再输入 `C:\xxx\android-ndk-r26d\ndk-build.cmd` 回车，即编译完成。

以上需自行替换为实际路径。



```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.19045.6466]
(c) Microsoft Corporation。保留所有权利。

C:\Users\abc>cd /d C:\Users\abc\Desktop\testModule\module_hello_world\jni

C:\Users\abc\Desktop\testModule\module_hello_world\jni>C:\Users\abc\android-ndk-r26d\ndk-build.cmd
Android NDK: APP PLATFORM not set. Defaulting to minimum supported version android 21.
Android NDK: WARNING:C:/Users/abc/Desktop/testModule/module_hello_world/jni/Android.mk:module_hello_world: non-system li
braries in linker flags: C:/Users/abc/Desktop/testModule/module_hello_world/jni/../../kernel_module_kit/lib/libkernel_mo
dule_kit_static.a
Android NDK: This is likely to result in incorrect builds. Try using LOCAL_STATIC_LIBRARIES
Android NDK: or LOCAL_SHARED_LIBRARIES instead to list the library dependencies of the
Android NDK: current module
[arm64-v8a] Compile++      : module_hello_world <= module_hello_world.cpp
[arm64-v8a] SharedLibrary  : libmodule_hello_world.so
[arm64-v8a] Install       : libmodule_hello_world.so => libs/arm64-v8a/libmodule_hello_world.so

C:\Users\abc\Desktop\testModule\module_hello_world\jni>
```

八、打包一个简单的 SKRoot 模块。

8.1 使用 Android NDK 编译生成 .so 文件，并将其打包为 **ZIP**

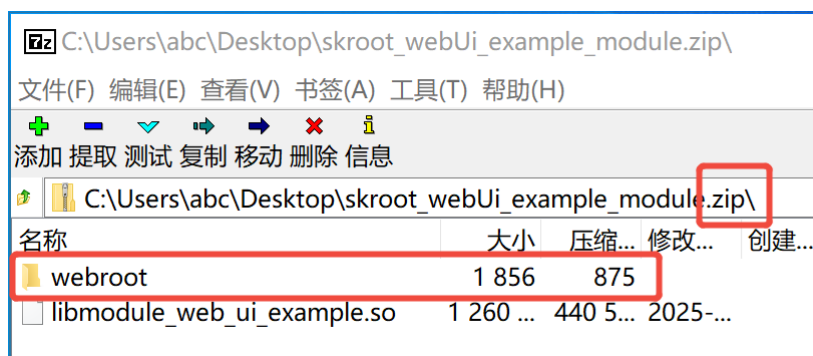
压缩文件，即可得到一个 SKRoot 模块。



参考项目: *module_hello_world*

九、打包一个具有 WebUI 页面的 SKRoot 模块。

- 9.1 **方法 1:** 新建 wwwroot 文件夹并在里面放入网页资源文件，
首页命名为 index.html，然后将 wwwroot 文件夹一起打
包进 **ZIP 压缩包**。



- 9.2 **方法 2:** 在源码处的“**模块名片**”添加

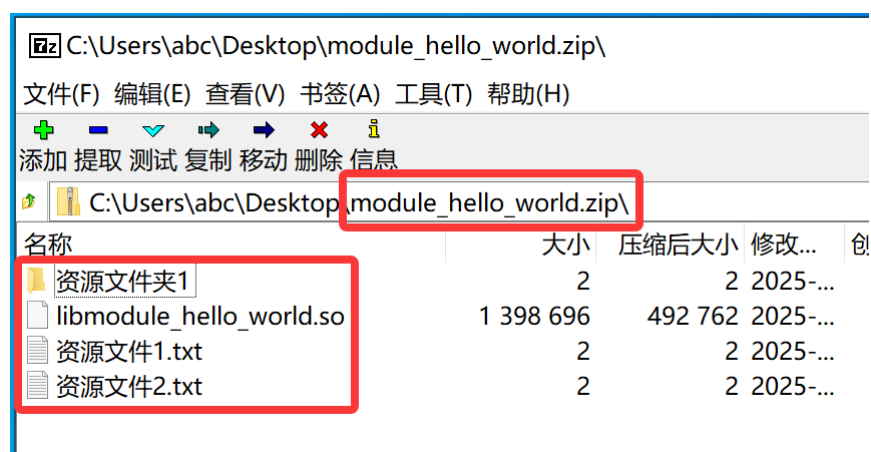
SKROOT_MODULE_WEB_UI，并填入 HTTP 回调响应类。

```
// SKRoot 模块名片
// 字段说明见 module_descriptor.h
SKROOT_MODULE_NAME("内置 WebUI Demo")
SKROOT_MODULE_VERSION("1.0.0")
SKROOT_MODULE_DESC("演示在模块中使用WebUI页面")
SKROOT_MODULE_AUTHOR("SKRoot")
SKROOT_MODULE_ID32("6080b19fb2db26c534af3051103f541f")
SKROOT_MODULE_WEB_UI(MyWebHttpHandler)
```

两种实现方法可并存。参考项目：*module_web_ui_example*

十、打包一个附带资源文件的 SKRoot 模块。

10.1 相关文件可直接随 **ZIP 模块包一起打包**。模块安装时，框架会自动释放文件到模块运行目录，并统一**设置为可读、可写、可执行权限（0777）**。



10.2 在编程中，入口 `skroot_module_main` 处使用

`module_private_dir` 变量即可访问此资源目录。

```
int skroot_module_main(const char* root_key, const char* module_private_dir) {
    return 0;
}
```